Combinatorial Zero-Shot Prompt Optimization via Ant Colony Optimization

Giammarco Vadacchino

Contributing authors: giammarco.vadacchino@studenti.unimi.it;

Abstract

Large Language Models (LLMs) are widely used for summarization, yet their performance strongly depends on prompt design, which is often manual and time consuming. In this work, i formulate prompt optimization as a structured combinatorial search problem and propose an Ant Colony Optimization (ACO) approach to automatically discover effective prompts. Prompts are represented as combinations of functional components and evaluated. I compare ACO-optimized prompts against human-written prompts.

1 Introduction

Recent work has explored automatic prompt optimization as an alternative to manual prompt engineering, treating prompt construction as a search or optimization problem. In this setting, prompts are treated as discrete objects and evaluated through black-box feedback obtained from the outputs of a Large Language Model (LLM), without requiring access to model internals or gradients. An assumption of these approaches is that a given task admits a large space of semantically valid prompts, each potentially inducing different model behaviors. Among these methods, EvoPrompt [1] proposes to optimize prompts using evolutionary algorithms, iteratively mutating and selecting candidate instructions based on task performance. Similarly, Automatic Prompt Engineer (APE) [2] formulates prompt generation as a black-box search problem, automatically proposing and evaluating prompts using external scoring functions. These works demonstrate that prompt optimization can be effectively automated and highlight the existence of a structured prompt space in which multiple prompt formulations may yield different levels of performance. However, most existing approaches rely on genetic or mutation-based search strategies and often operate on prompts as unique text sequences, without explicitly modeling their internal compositional structure. In practice, prompts are frequently composed of multiple functional components—such

as role instructions, task descriptions, stylistic constraints, and output formatting requirements—suggesting that prompt optimization can be formulated as a structured combinatorial problem. In this work, i model prompt engineering for summarization task as a path-based combinatorial optimization task. Prompts are decomposed into modular components and represented as paths in a directed acyclic graph (DAG), defining a discrete and structured prompt space. To explore this space, i used Ant Colony Optimization (ACO) [3], a population-based meta-heuristic algorithm well suited for graph-based optimization. By treating the LLM as a black-box function and using automatic evaluation metrics as feedback, this approach enables the discovery of effective prompt configurations.

2 Methodology

2.1 Goal of the Project

This project aims to explore the feasibility of automating prompt design for text summarization using a meta-heuristic optimization approach. It focuses on the development and evaluation of an Ant Colony Optimization based pipeline for identifying effective prompt configurations for Large Language Models. A key objective is to systematically assess the performance of automatically optimized prompts in comparison with human-crafted prompts, ensuring a fair and reproducible experimental setup.

2.2 Formal Definition of the Problem

Let $D = \{d_1, d_2, ..., d_n\}$ be a set of source document, and let $S = \{s_1, s_1, ..., s_n\}$ be the corresponding reference summaries. The task of prompt optimization can be formally defined as finding a prompt p^* from a structure prompt research space P that maximizes the expected quality of generated summaries:

$$p^* = \arg\max_{p \in \mathcal{P}} \mathbb{E}_{\hat{s} \sim \text{LLM}(d,p)} \big[Q(\hat{s}, s) \big] \tag{1}$$

where $\hat{s}$ denotes a summary generated by the LLM given document d and prompt p, and $Q(s, \hat{s})$ is a fitness function. In this project the fitness function aggregates lexical (e.g,ROUGE), semantic (e.g BERTScore), and faithfulness (e.g NLI-based) metrics. The objective is to identify prompts that consistently yield high-quality,semantically,relevant and factually faithful summaries.

2.3 ACO Prompt Optimization

The main challenge in applying Ant Colony Optimization is that the optimization problem must be formulated as a path-based graph. To this end, i represent structured zero-shot prompts as paths in a Directed Acyclic Graph (DAG). Rather than treating a prompt as a monolithic text string, i decompose it into a sequence of functional components, such as role instructions, task descriptions, stylistic constraints, or output

formatting guidelines. Each component corresponds to a distinct level of the DAG, and each level contains multiple nodes representing alternative ways of expressing the same functional instruction. These alternatives are semantically valid variants that differ in phrasing or emphasis, allowing the optimization process to explore different prompt formulations. Variants for each level are automatically generated using a large language model and subsequently filtered using cosine similarity to ensure sufficient diversity. By construction, edges in the DAG connect nodes belonging to consecutive levels, and each path from the start node to the final level selects exactly one variant per component. As a result, every complete path uniquely defines a valid structured prompt. This representation naturally maps the prompt optimization problem to a combinatorial path-finding task, enabling the application of Ant Colony Optimization to efficiently explore a rich and structured prompt space. After we build the DAG we can apply ACO that can be divided in three phases:

1. **Initialization**: in this phase each edge is assigned an initial pheromone level, some parameters are set such as pheromone influence (α), evaporation rate and pheromone scaling factor (Q).
2. **Path construction**: in this phase each ant starts at the start node of the DAG, then he cross the graph and build a path (that represents a prompt) probabilistically,the probability of choosing the next node depends on pheromone level. When the ants reach the last level of the DAG, the path is serialized into a prompt and the the prompt is ready to be evaluated.
3. **Pheromone Update**: in this phase pheromones evaporate on all edge to avoid saturation. Edges of path found by ants are reinforced according to the chosen strategy:
   - **Classic**: each ant reinforces the edges it traveled, with path with better fitness value leaving more pheromone. This approach spreads reinforcement across all discovered paths, but good (path with high fitness value) paths naturally get a stronger signal.
   - **Elitist**: works like the classic system, but the global path receives an extra boost of pheromones. This accelerates convergence by making the best path more likely to be chosen in future iterations.
   - **Rank-Based**: only the top K paths (the K paths with highest fitness value in the current iteration) contribute to the pheromone update, the higher a path ranks, the more pheromone it deposits.
   - **Max-Min**: only the best path in the current iteration updates pheromones, upper and lower bounds are imposed on the pheromone levels to prevent any edge from becoming too dominant (too much pheromone) or too weak (too less pheromone).
   - **Best-Worst**: combines reinforcement of the best path with the reduction of pheromones on the worst path. By decreasing the attractiveness of poor paths, it reduces the chance they are chosen in the future.

All these phases are repeated for a fixed number of iterations. Through pheromone reinforcement, the ants are progressively guided toward prompt configurations that should yield higher fitness values.

3 Experimental Setup

3.1 Dataset

Experiments were conducted on a subset of the CNN/DailyMail news summarization dataset, obtained from a publicly available Kaggle release (https://www.kaggle.com/datasets/gowrishankarp/newspaper-text-summarization-cnn-dailymail?resource=download, **last access**: 11/01/2026). Due to computational constraints and input length limitations of the adopted language model, a fixed set of 16 document–summary pairs was selected and used across all experiments. Source documents were filtered by length to ensure compatibility with the maximum input size of the model (512 tokens), resulting in articles of approximately 320–370 tokens.

3.2 Model and Generation Settings

All summaries were generated using T5-base, a pretrained encoder–decoder language model with approximately 220 million parameters, commonly used for summarization tasks. The model was used in a zero-shot setting, without any task-specific fine-tuning, in order to isolate the effect of prompt optimization from model adaptation. Text generation was performed using fixed decoding parameters across all experiments to ensure fair comparison between different prompt configurations. In particular, the temperature was set to 0.3 to limit output variability while preserving a degree of stochasticity.

3.3 Instantiation of the Prompt Space

Following the structured formulation introduced in Section 2.3, the prompt space was instantiated as a directed acyclic graph composed of multiple prompt levels. For each level, 15 alternative instruction variants were generated using GPT5 (GPT5 was used in order to have better variants but the pipeline allows the use of other models too) and filtered through cosine similarity to ensure sufficient diversity. The resulting DAG defines a discrete combinatorial prompt space, where each path corresponds to a valid structured prompt.

3.4 ACO configuration

The Ant Colony Optimization algorithm was initialized with 4 ants and executed for 4 iterations. Each ant constructs a candidate prompt by traversing the prompt space graph as defined in Section 2.3. Pheromone values were initialized uniformly across edges, and standard pheromone evaporation was applied at each iteration to prevent premature convergence. Due to limited computational resources, the algorithm's hyperparameters were not fine-tuned.

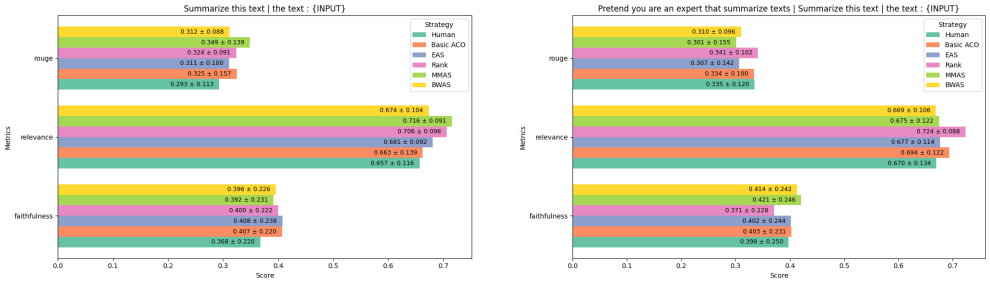
3.5 Evaluation Metrics and Fitness Function

Prompt quality was evaluated using a combination of lexical, semantic, and faithfulness-oriented metrics. Lexical overlap between generated and reference summaries was measured using ROUGE [4], computed as a weighted combination of

ROUGE-1, ROUGE-2, and ROUGE-L F1 scores. Semantic similarity was assessed using cosine similarity between sentence embeddings produced by the paraphrase-MiniLM-L6-v2 model, which captures meaning-level correspondence and paraphrasing phenomena beyond surface overlap. Faithfulness was evaluated through a Natural Language Inference (NLI)–based approach, using the pretrained valhalla/distilbart-mnli-12-1 model to classify the relationship between generated summaries and source documents. The overall fitness score used by the optimization algorithm is defined as a weighted sum of the three metrics, with weights of 0.3 for ROUGE, 0.2 for semantic similarity, and 0.5 for faithfulness. This composite fitness function encourages the selection of prompts that balance content relevance, semantic adequacy, and factual consistency.

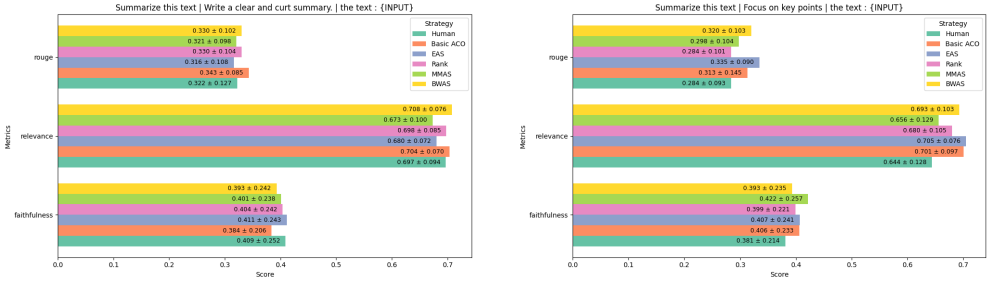
4 Results and Discussions

The following figures report the evolution of the fitness score across ACO iterations for different initial prompts.



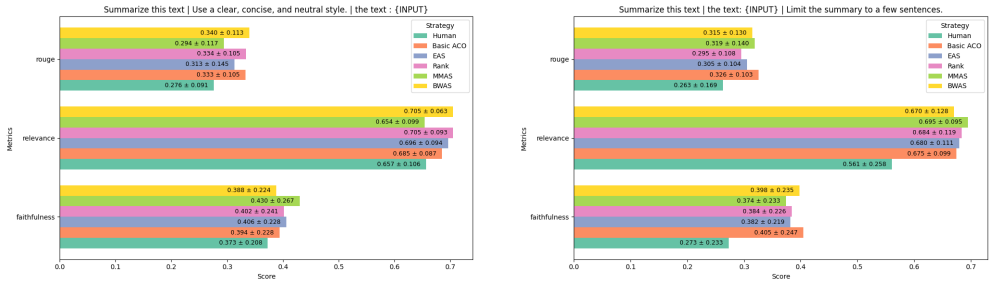
(a) Basic Prompt (b) Role Instruction + Basic Prompt

Fig. 1: ACO performance using different initial prompts.



(a) Fidelity Constraint + Basic Prompt (b) Salience Guidance + Basic Prompt

Fig. 2: ACO performance using different initial prompts.



(a) Style Constraint + Basic Prompt

(b) Length Constraint + Basic Prompt

Fig. 3: ACO performance using different initial prompts.

The experimental results reported in Figures 1–3 show the metrics of the best prompt identified by Ant Colony Optimization over four iterations, with each figure corresponding to a different initial prompt.

Overall, the trends observed across the figures are highly consistent, with only minor variations in absolute fitness values between different initial prompts. This indicates that the optimization process is largely insensitive to the choice of the starting prompt and is instead driven by the structure of the prompt space and the stochastic exploration induced by ACO. Across all experiments, a modest improvement over the initial prompt can be observed, suggesting that the algorithm is capable of identifying prompt configurations that perform slighty better than the manually specified starting point.

However, the improvements observed are relatively small, and no clear convergence can be identified given the limited number of iterations. Instead of steadily increasing, fitness values fluctuate across iterations. This behavior is expected due to the constrained experimental setup, with only four ants and four iterations, which keeps the algorithm mainly in an exploratory phase rather than allowing full convergence.

The reported results do not correspond to a converged ACO solution. Due to the limited search budget, the pheromone reinforcement mechanism cannot be fully exploited, and the best prompts identified should be interpreted as intermediate solutions rather than globally optimal ones. This limitation is directly linked to the high computational cost of prompt evaluation, as each candidate prompt requires multiple calls to the large language model.

Despite these constraints, the results provide evidence that modeling prompt optimization as a structured combinatorial problem can be a viable approach. Even under a shallow search budget, ACO is able to navigate the prompt DAG and identify prompt configurations that are competitive with manually designed prompts.

5 Conclusions and Future Work

Experimental results show that, even with strong computational limitations and without full convergence, the proposed method can find prompt configurations that perform similarly to manually designed prompts. Although the improvements are modest and

no clear convergence is observed, the results demonstrate that graph-based prompt optimization using meta-heuristic search is feasible. At the same time, they reveal important limitations related to the high computational cost of LLM evaluations and the limited depth of the search. Future work will focus on scaling the approach to larger datasets and longer optimization runs by taking advantage of the parallel nature of Ant Colony Optimization. In addition, testing alternative graph representations for structured prompts and using more efficient evaluation strategies could improve both performance and computational efficiency.

References

[1] Qingyan Guo, Rui Wang, Junliang Guo, Bei Li, Kaitao Song, Xu Tan, Guoqing Liu, Jiang Bian, Yujiu Yang, *EvoPrompt: Connecting LLMs with Evolutionary Algorithms Yields Powerful Prompt Optimizers*, arXiv:2309.08532, 2025, https://arxiv.org/abs/2309.08532

[2] Yongchao Zhou, Andrei Ioan Muresanu, Ziwen Han, Keiran Paster, Silviu Pitis, Harris Chan, Jimmy Ba, *Large Language Models Are Human-Level Prompt Engineers*, arXiv:2211.01910, 2023, https://arxiv.org/abs/2211.01910

[3] Marco Dorigo, Luca M. Gambardella, *Ant colony system: a cooperative learning approach to the traveling salesman problem*, *IEEE Transactions on Evolutionary Computation*, vol. 1, no. 1, pp. 53–66, 1997, doi:10.1109/4235.585892

[4] Chin-Yew Lin, *ROUGE: A Package for Automatic Evaluation of Summaries*, in *Text Summarization Branches Out*, Association for Computational Linguistics, Barcelona, Spain, July 2004, pp. 74–81, https://aclanthology.org/W04-1013/